Lab Handout 1-H
Field-Programmable Gate Array (FPGA) Universal Asynchronous Receiver/Transmitter (UART) Ingestion and End-to-End Data Flow

Capstone — Module 1: Radio Frequency (RF) Hardware Foundations

---

1 Objective

Implement a minimal UART ingestion path on the Nexys A7-100T that receives the 43-byte forwarding frames from the STM32 receiver (LH1-G), detects the start delimiter, extracts the beacon ID, and displays it on the seven-segment display. This closes the Module 1 end-to-end data path: **beacon → RF link → receiver → FPGA**.

2 Prerequisites

- Lab Handouts 1-E (Vivado toolchain), 1-F (beacon), and 1-G (receiver) complete.
- The receiver node from LH1-G must be producing valid frames on `USART3 TX` (validated by the Python parser).

3 Architecture

Block	Function
`uart_rx`	115200 8N1 UART receiver (oversampled)
`byte_fifo`	Small 16-byte First In, First Out (FIFO) between `uart_rx` and frame detector
`frame_detector`	Synchronize on `0x7E` start, capture 43 bytes, expose beacon ID
`seg7_driver`	Drive the four right-most digits with the beacon ID in hex

[Figure placeholder]

`LH1H_block_diagram.png`

FPGA ingestion block diagram: `uart_rx` → `byte_fifo` → `frame_detector` → `seg7_driver`. Key signals (`rx_strobe`, `beacon_id`, `new_frame`) are labeled on the data path.

Figure 1: FPGA ingestion block diagram: `uart_rx` → `byte_fifo` → `frame_detector` → `seg7_driver`. Key signals (`rx_strobe`, `beacon_id`, `new_frame`) are labeled on the data path.

4 Wiring

Connect the receiver's `USART3 TX` pin to one of the Nexys A7 PMOD input pins. Common ground is essential.

- STM32 `USART3 TX` (`PC4`) → Nexys A7 `JA[0]` (pin `C17`, check the PMOD pin mapping in the Nexys A7 reference manual).
- STM32 `GND` → Nexys A7 `GND` (`JA` PMOD pin 5 or 11).

[Figure placeholder]

`LH1H_pmod_wiring.png`

STM32 `USART3 TX` (`PC4`) wired to Nexys A7 `JA[0]` (pin `C17`) with common ground. Annotated photo clarifies which PMOD connector is in use.

Figure 2: STM32 `USART3 TX` (`PC4`) wired to Nexys A7 `JA[0]` (pin `C17`) with common ground. Annotated photo clarifies which PMOD connector is in use.

Voltage Levels

The STM32 drives 3.3 V logic and the Nexys A7 PMOD pins are 3.3 V-tolerant (in fact, 3.3 V native). No level shifter is required. Do *not* wire the STM32 to an LVCMOS18 bank by mistake — double-check which PMOD you are using.

5 Procedure

5.1 Step 1 — Project setup

1. In Vivado, create a new project targeting `xc7a100tcsg324-1` (or reopen the LH1-E project and add the new sources alongside `loopback_top.vhd`).
2. Add a new top-level entity `ingest_top.vhd` with:
   - `clk100` (100 MHz input from the Nexys A7 oscillator).
   - `rst_n` (active-low reset tied to a pushbutton).
   - `uart_rx` (single input from the PMOD pin).
   - `seg` (7 outputs for segments) and `an` (8 outputs for digit anodes).
3. Add the pruned `Nexys-A7-100T-Master.xdc` with the relevant pins uncommented.

5.2 Step 2 — UART receiver

1. Write or reuse a textbook 115200 8N1 UART receiver. Clock: 100 MHz, so the baud-tick divisor is $100\,000\,000/115\,200 \approx 868$. Oversample by 16 and sample at bit midpoints.
2. Expose two outputs: `rx_data[7:0]` and `rx_strobe` (one-cycle pulse on byte received).

5.3 Step 3 — Byte FIFO

1. Instantiate a small synchronous FIFO (16 entries $\times$ 8 bits is plenty). You can hand-roll it in VHSIC Hardware Description Language (VHDL) or instantiate a Xilinx FIFO primitive — either is fine for this handout.
2. Push on `rx_strobe`; the frame detector pulls one byte per state tick.

5.4 Step 4 — Frame detector Finite State Machine (FSM)

States: `IDLE`, `LEN`, `PAYLOAD`, `DONE`.

1. `IDLE`: pop bytes and discard until a byte equals `0x7E`. Transition to `LEN`.
2. `LEN`: pop one byte. Expect `41` (0x29). If it doesn't match, return to `IDLE`. Otherwise, transition to `PAYLOAD`.
3. `PAYLOAD`: pop 41 more bytes into an internal register bank. The frame layout is: delimiter at 0, length (`0x29`) at 1, metadata (timestamp, Received Signal Strength Indicator (RSSI), Signal-to-Noise Ratio (SNR), freq. error) at 2–9, embedded 32-byte beacon payload at 10–41, frame Cyclic Redundancy Check (CRC)-8 at 42. The beacon ID is beacon byte 2, i.e. frame offset 12. Capture it into a register `beacon_id[7:0]`.
4. `DONE`: assert a `new_frame` pulse for one cycle, then return to `IDLE`.

For Week 4 scope, ignore the frame CRC-8 and embedded beacon CRC-16 checks inside the FPGA. Module 2 will add both.

[Figure placeholder]

`LH1H_frame_detector_fsm.png`

Frame-detector FSM: `IDLE` → `LEN` → `PAYLOAD` → `DONE`, with transition conditions annotated on each edge.

Figure 3: Frame-detector FSM: `IDLE` → `LEN` → `PAYLOAD` → `DONE`, with transition conditions annotated on each edge.

5.5 Step 5 — Seven-segment driver

1. Multiplex the four right-most digits at ∼1 kHz per digit (use a clock divider from 100 MHz).
2. Display the captured `beacon_id` in hex on digits `[1:0]`. Leave digits `[3:2]` blank or use them to show a simple frame counter that increments on `new_frame`.

5.6 Step 6 — Synthesize, implement, program

1. Follow the same flow as LH1-E: Synthesis, Implementation, Generate Bitstream, Program.
2. Verify zero timing-constraint violations in the implementation report.

[Figure placeholder]

`LH1H_timing_report.png`

Vivado implementation timing summary with positive WNS. Any negative slack must be resolved before programming the board.

Figure 4: Vivado implementation timing summary with positive WNS. Any negative slack must be resolved before programming the board.

5.7 Step 7 — End-to-end run

1. Power on the beacon(s) from LH1-F and the receiver from LH1-G. The STM32 receiver should be `printf`ing debug output over its virtual COM port.
2. Power on the Nexys A7 and program the bitstream.
3. Observe the seven-segment display. It should alternate between `01` and `02` as each beacon's frames arrive.
4. Introduce a deliberate perturbation: turn off Beacon A. The display should lock to `02` only. Turn it back on. It should resume alternating.

[Figure placeholder]

`LH1H_seg7_display.jpg`

Module 1 capstone shot: Nexys A7 seven-segment display showing a live beacon ID captured from the full ***beacon*** → ***RF link*** → ***receiver*** → ***FPGA*** *chain.*

Figure 5: Module 1 capstone shot: Nexys A7 seven-segment display showing a live beacon ID captured from the full **beacon** → **RF link** → **receiver** → **FPGA** chain.

6 Verification Checklist

- ☐ `uart_rx` correctly decodes bytes at 115200 baud (verify with ILA or test bench).
- ☐ Frame detector synchronizes on the `0x7E` delimiter and captures 43-byte frames without losing alignment.
- ☐ Seven-segment display shows `01`/`02` matching the live beacon IDs.
- ☐ Removing and restoring a beacon causes the display to respond within 2 seconds.
- ☐ Vivado implementation report: zero timing violations.

Troubleshooting

- **No display change**: verify the PMOD pin is correct in the `.xdc` and that the STM32 ground is tied to the Nexys A7 ground.
- **Garbage bytes**: baud divisor wrong. Recompute against the actual `clk100` frequency.
- **Frame detector stuck in `IDLE`**: length byte mismatch. Confirm the receiver is sending `0x29` as byte 1.
- **Display flickers unreadably**: multiplex rate too slow (increase per-digit frequency to $\geq$500 Hz).

Lab Handout 1-H Deliverable — and Module 1 Final Deliverable

1. All VHDL sources: `uart_rx`, `byte_fifo`, `frame_detector`, `seg7_driver`, `ingest_top`.
2. Pruned `.xdc` constraint file.
3. Vivado utilization and timing reports.
4. Short video (30–60 s) of the complete end-to-end system running: beacons transmitting, receiver forwarding, Nexys A7 seven-segment display updating with beacon IDs.
5. One-page Module 1 retrospective: what worked, what didn't, what you would change before Module 2.

Glossary

CRC	Cyclic Redundancy Check	3
FIFO	First In, First Out	1, 3
FPGA	Field-Programmable Gate Array	1, 2, 3, 5
FSM	Finite State Machine	3, 4
RF	Radio Frequency	1, 5
RSSI	Received Signal Strength Indicator	3
SNR	Signal-to-Noise Ratio	3
UART	Universal Asynchronous Receiver/Transmitter	1, 3
VHDL	VHSIC Hardware Description Language	3, 6